

IA - KR

Rezolvari dezvoltate pentru examen

Modele de examen, subiecte anterioare, seminare si Curs 14

Ce contine materialul:

- sabloane de raspuns pentru problemele care se repeta la examen;
- rezolvari detaliate pentru subiectul 27 ianuarie 2026;
- rezolvari detaliate pentru subiectul 2 februarie 2024;
- rezolvari detaliate pentru subiectul 4 februarie 2025;
- rezolvari pentru modelul de subiect decembrie 2023;
- explicatii despre d-separare si retele Bayesiene din Cursul 14;
- fisa rapida de invatat inainte de examen.

Material de invatat rapid: accent pe pasi, justificari si formulari de examen.

Contents

1	Cum folosești materialul	2
2	Sabloane generale de răspuns	2
2.1	Sablon pentru formularea unei probleme de căutare	2
2.2	Sablon pentru căutare pe graf	2
2.3	Sablon pentru Minimax și Alpha-Beta	3
2.4	Sablon pentru PSC/CSP	3
2.5	Sablon pentru rețele Bayesiene	3
3	Subiect 27 ianuarie 2026 - rezolvare dezvoltată	4
3.1	Problema 1 - Melcul și unicornul în labirint	4
3.2	Problema 2 - DFS, BFS, Greedy și A*	5
3.3	Problema 3 - Colorare hartă cu backtracking și forward checking	6
3.4	Problema 4 - Minimax și Alpha-Beta	7
3.5	Problema 5 - Rețea Bayesiană G și T	8
4	Subiect 2 februarie 2024 - rezolvare dezvoltată	9
4.1	Problema 1 - Roboții Memo și Pic	9
4.2	Problema 2 - DFS, BFS, UCS, euristici	9
4.3	Problema 3 - Minimax și Alpha-Beta	10
4.4	Problema 4 - N-regine, forward checking și arc consistency	11
5	Subiect 4 februarie 2025 - rezolvare dezvoltată	12
5.1	Problema 1 - Robot care trebuie să treacă prin G_1 și G_2	12
5.2	Problema 2 - DFS, BFS, Greedy, A* și consistență	12
5.3	Problema 3 - Minimax cu valori lipsă E și K	14
5.4	Problema 4 - Sudoku ca PSC	15
5.5	Problema 5 - Rețeaua Bayesiană Wet Grass	15
6	Model subiect decembrie 2023 - rezolvare dezvoltată	16
6.1	Problema 1 - Robot pe drum alb/negru	16
6.2	Problema 2 - Graf cu DFS, Greedy și A*	16
6.3	Problema 3 - Conectează-3	17
6.4	Problema 4 - Euristică PSC în Sudoku	18
7	Curs 14 - D-separare și rețele Bayesiene	19
7.1	Cele trei triplete fundamentale	19
7.1.1	1. Lant cauzal	19
7.1.2	2. Cauza comună	19
7.1.3	3. Efect comun / collider	19
7.2	Algoritmul de D-separare	19
7.3	Exemplu: $P \rightarrow T \leftarrow C, T \rightarrow A$	19
7.4	Exemplu extins: $P_s \rightarrow P, P \rightarrow U, P \rightarrow T, C \rightarrow T, T \rightarrow A$	20
8	Fisa finală pentru examen	22

1 Cum folosești materialul

Examenul de KR are un tipar foarte stabil. Aproape toate subiectele se încadrează în următoarele familii:

Tip problema	Ce se cere	Sablon de rezolvare
Formulare problema de cautare Cautare pe graf	stare, stare initiala, scop, actiuni, succesor, numar de stari, euristica DFS, BFS, UCS, Greedy, A*	identifici informatia care se schimba si o pui in stare scrii frontiera pas cu pas, cu g, h, f unde e cazul
Euristici	admisibilitate si consistenta	compari $h(n)$ cu $h^*(n)$, respectiv verifici $h(n) \leq c(n, n') + h(n')$
Jocuri adversariale	Minimax, Alpha-Beta	calculezi de jos in sus; MAX ia maxim, MIN ia minim; tai cand $\alpha \geq \beta$
PSC/CSP	colorare, Sudoku, N-regine	definesti X, D, C , aplici forward checking sau consistenta arcelor
Rețele Bayesiene	factorizare, distributie comuna, independenta conditionata	inmultesti probabilitatile conform grafului; pentru independenta folosesti d-separare

Important: La examen nu este suficient sa scrii doar raspunsul final. Punctele vin din pasii intermediari: frontiera, domeniile eliminate, valorile minimax, probabilitatile intermediare si justificarea.

2 Sabloane generale de raspuns

2.1 Sablon pentru formularea unei probleme de cautare

1. **Stare:** $s = \dots$ retine exact informatia necesara pentru a continua problema.
2. **Stare initiala:** $s_0 = \dots$ data in enunt.
3. **Stare finala:** conditie de scop, nu neaparat o singura stare.
4. **Actiuni:** mutarile permise dintr-o stare.
5. **Functie succesor:** $Succ(s, a) = s'$ daca actiunea este valida.
6. **Cost:** de obicei 1 pe mutare, daca enuntul nu spune altceva.
7. **Factor de ramificare:** numarul maxim de succesor.
8. **Numar de stari:** produsul posibilitatilor pentru componentele starii.
9. **Euristica admisibila:** estimare care nu supraestimeaza costul real.

2.2 Sablon pentru cautare pe graf

Algoritm	Alege urmatorul nod dupa	Ce scrii in rezolvare
DFS	cel mai recent drum din frontiera	frontiera ca stiva; mergi pe prima ramura lexicografica
BFS	drumul cu cea mai mica adancime	frontiera pe niveluri; la varianta eficienta te opresti cand generezi scopul
UCS	costul $g(n)$ minim	notezi costul fiecarui drum partial
Greedy	euristica $h(n)$ minima	ignori costul parcurs deja; alegi cel mai promitator nod

A*

 $f(n) = g(n) + h(n)$ minimcalculezi g, h, f și actualizezi drumurile mai bune

2.3 Sablon pentru Minimax și Alpha-Beta

Minimax: calculez de jos în sus. La MAX pun maximul copiilor. La MIN pun minimul copiilor. Mutarea aleasă este copilul rădăcinii care dă valoarea finală.

Alpha-Beta: pornesc cu $\alpha = -\infty$, $\beta = +\infty$. La MAX actualizez α . La MIN actualizez β . Tai când $\alpha \geq \beta$. Alpha-Beta nu schimbă valoarea Minimax, doar reduce nodurile vizitate.

2.4 Sablon pentru PSC/CSP

Modelăm problema ca $PSC = (X, D, C)$, unde X sunt variabilele, D sunt domeniile, iar C sunt constrangerile. O stare este o asignare parțială. Starea inițială este asignarea vidă. Starea scop este o asignare completă și consistentă. Pentru forward checking, după fiecare asignare eliminăm valorile incompatibile din domeniile vecinilor.

2.5 Sablon pentru rețele Bayesiene

Dacă graful este $X_1, \dots, X_n$, atunci distribuția comună se factorizează astfel:

$$P(X_1, \dots, X_n) = \prod_i P(X_i \mid \text{Parinti}(X_i)).$$

Pentru independența condiționată folosim d-separarea: dacă toate drumurile neorientate între două variabile sunt inactive/blocate de evidență, atunci independența este garantată.

3 Subiect 27 ianuarie 2026 - rezolvare dezvoltată

3.1 Problema 1 - Melcul și unicornul în labirint

Enunț pe scurt. Doi agenți, M și U , sunt într-un labirint 6×14 . Fiecare poate merge stanga, dreapta, sus, jos sau poate sta. Se mișcă simultan. Scopul este să ajungă pe aceeași celulă în număr minim de mutații.

a) Reprezentarea stării, starea inițială, starea finală, acțiunile

Rezolvare explicată. Pentru că avem doi agenți, starea trebuie să țină poziția fiecăruia:

$$s = ((x_M, y_M), (x_U, y_U)).$$

Coordonatele sunt în dreptunghiul 6×14 , deci $x \in \{1, \dots, 14\}$ și $y \in \{1, \dots, 6\}$, dar pozițiile obstacolelor nu sunt valide.

Starea inițială este:

$$s_0 = ((1, 6), (14, 1)).$$

O stare finală este orice stare în care cei doi agenți ocupă aceeași celulă:

$$s_f = ((x, y), (x, y)).$$

Acțiunile fiecărui agent sunt:

$$A_M = A_U = \{\text{stanga}, \text{dreapta}, \text{sus}, \text{jos}, \text{stat}\}.$$

Pentru că agenții se mișcă simultan, acțiunea globală este:

$$A = A_M \times A_U.$$

Exemplu:

$$(\text{stanga}_M, \text{jos}_U)$$

este acțiunea în care M merge la stanga și U merge jos în același pas.

b) Factor maxim de ramificare

Fiecare agent are maxim 5 acțiuni. Pentru doi agenți simultan:

$$b_{\max} = 5 \cdot 5 = 25.$$

Nu toate cele 25 sunt valide în fiecare stare, deoarece pot exista margini sau obstacole, dar întrebarea cere valoarea maximă ignorând forma labirintului.

c) Număr de stări

Labirintul are:

$$6 \cdot 14 = 84 \text{ celule.}$$

Dacă sunt 25 obstacole, avem:

$$84 - 25 = 59 \text{ celule libere.}$$

Fiecare agent poate fi în oricare dintre cele 59 de celule libere, iar scopul permite ca amândoi să stea pe aceeași celulă. Deci:

$$|S| = 59^2 = 3481.$$

d) Euristica admisibila

Distanța Manhattan dintre agenti este:

$$D = |x_M - x_U| + |y_M - y_U|.$$

Intr-o mutare simultana, cei doi se pot apropia cu maximum 2 unitati Manhattan. O euristica informativa este:

$$h(s) = \left\lceil \frac{|x_M - x_U| + |y_M - y_U|}{2} \right\rceil.$$

Este admisibila deoarece ignora obstacolele si presupune ca ambii se pot apropia optim. Ignorarea obstacolelor nu poate mari estimarea, deci h nu supraestimeaza costul real.

3.2 Problema 2 - DFS, BFS, Greedy si A*

Graful are noduri S, A, B, C, D, G si euristica:

$$h(S) = 3, h(A) = 2, h(B) = 1, h(C) = 4, h(D) = 2, h(G) = 0.$$

Muchiile relevante sunt:

$$\begin{aligned} S \rightarrow A(1), \quad S \rightarrow G(12), \quad A \rightarrow B(3), \quad A \rightarrow C(1), \\ B \rightarrow D(3), \quad C \rightarrow D(1), \quad C \rightarrow G(2), \quad D \rightarrow G(3). \end{aligned}$$

a) DFS

Rezolvare explicata. Folosim ordinea lexicografica. Frontiera, ca drumuri partiale:

$$\begin{aligned} &S \\ &S - A, S - G \\ &S - A - B, S - A - C, S - G \\ &S - A - B - D, S - A - C, S - G \\ &S - A - B - D - G, S - A - C, S - G. \end{aligned}$$

Prima solutie gasita de DFS este:

$$\boxed{S - A - B - D - G}.$$

b) BFS eficient

BFS exploreaza pe niveluri si, in implementarea eficienta, se opreste cand genereaza scopul.

$$\begin{aligned} &S \\ &S - A, S - G. \end{aligned}$$

Scopul este generat la adancimea 1, deci:

$$\boxed{S - G}.$$

c) Admisibilitatea euristicii

O euristica este admisibila daca $h(n) \leq h^*(n)$ pentru orice nod. Din C catre G exista muchie directa de cost 2, deci:

$$h^*(C) = 2.$$

Dar:

$$h(C) = 4 > 2.$$

Deci h nu este admisibila.

d) Greedy

Greedy alege minimul după h .

$$S(3) \\ S - A(2), S - G(0).$$

Alege G , deci:

$$\boxed{S - G}.$$

e) A*

A* folosește:

$$f(n) = g(n) + h(n).$$

Pasii:

$$S : 0 + 3 = 3.$$

După expandarea lui S :

$$S - A : 1 + 2 = 3, \quad S - G : 12 + 0 = 12.$$

Expandăm $S - A$:

$$S - A - B : 4 + 1 = 5, \quad S - A - C : 2 + 4 = 6, \quad S - G : 12.$$

Expandăm $S - A - B$:

$$S - A - B - D : 7 + 2 = 9, \quad S - A - C : 6, \quad S - G : 12.$$

Expandăm $S - A - C$:

$$S - A - C - D : 3 + 2 = 5, \quad S - A - C - G : 4 + 0 = 4, \quad S - A - B - D : 9, \quad S - G : 12.$$

Următorul nod minim este $S - A - C - G$, deci:

$$\boxed{S - A - C - G, \text{ cost} = 4}.$$

3.3 Problema 3 - Colorare harta cu backtracking si forward checking

Avem variabilele A, B, C, D, E, F , fiecare cu domeniul:

$$\{\text{rosu}, \text{verde}, \text{albastru}\}.$$

Constrangerea: două regiuni vecine nu pot primi aceeași culoare.

a) Graful de constrangeri

Graful are muchii între regiunile vecine. Din rezolvare, gradele duc la ordinea:

$$B - D - F - A - C - E.$$

b) Backtracking + forward checking

Initial:

$$D(A) = D(B) = D(C) = D(D) = D(E) = D(F) = \{R, V, A\},$$

unde $R = \text{rosu}$, $V = \text{verde}$, $A = \text{albastru}$.

Alegem:

$$B = R.$$

Eliminăm R din domeniile vecinilor lui B :

$$D(D) = D(F) = D(A) = D(C) = \{V, A\}.$$

Alegem:

$$D = V.$$

Eliminăm V din domeniile vecinilor lui D :

$$D(F) = \{A\}, \quad D(C) = \{A\}, \quad D(E) = \{R, A\}.$$

Alegem:

$$F = A.$$

Prin forward checking:

$$A = V, \quad E = R.$$

O prima soluție este:

$$B = R, \quad D = V, \quad F = A, \quad A = V, \quad C = A, \quad E = R.$$

Observăm structura:

$$B = E, \quad D = A, \quad F = C.$$

Alegem culori diferite pentru B și D : $3 \cdot 2 = 6$ variante. Restul sunt forțate. Deci sunt:

$$6 \text{ soluții.}$$

3.4 Problema 4 - Minimax și Alpha-Beta

Radacina este un nod MIN. Copiii sunt noduri MAX. Parcurgerea este de la stânga la dreapta.

a) Minimax

Subarborele stâng:

- frunza 13;
- nodul MIN cu copii 7 și nod MAX cu frunze 15, 3, 4; nodul MAX are valoarea 15, deci nodul MIN are $\min(7, 15) = 7$;
- frunza 10.

Nodul MAX din stânga are:

$$\max(13, 7, 10) = 13.$$

Subarborele drept:

- nod MIN cu frunza 16, valoare 16;
- nod MIN cu frunze 5, 27, valoare 5.

Nodul MAX din dreapta are:

$$\max(16, 5) = 16.$$

Radacina MIN are:

$$\min(13, 16) = 13.$$

Valoarea Minimax în radacina este:

$$13.$$

b) Alpha-Beta

Parcurgem de la stânga la dreapta. După ce subarborele stâng da valoarea 13, radacina MIN are $\beta = 13$. În subarborele drept, primul copil produce valoarea 16 pentru MAX; deoarece MAX are deja o valoare care nu poate face nodul drept mai mic decât ce are MIN nevoie, se pot elimina ramuri. În rezolvarea modelului, ramurile eliminate sunt:

$$h \text{ și } f.$$

Alpha-Beta returnează aceeași valoare ca Minimax, dar evita calculul unor frunze.

3.5 Problema 5 - Retea Bayesiană G și T

Avem rețeaua $G \rightarrow T$, unde G = gripa, T = tuse.

$$P(G = 1) = 0.1, \quad P(T = 1 \mid G = 1) = 0.8, \quad P(T = 1 \mid G = 0) = 0.3.$$

a) Distribuția comună $P(G, T)$

Completăm:

$$P(G = 0) = 0.9, \quad P(T = 0 \mid G = 1) = 0.2, \quad P(T = 0 \mid G = 0) = 0.7.$$

Folosim:

$$P(G, T) = P(G)P(T \mid G).$$

Calcul:

$$P(G = 1, T = 1) = 0.1 \cdot 0.8 = 0.08,$$

$$P(G = 1, T = 0) = 0.1 \cdot 0.2 = 0.02,$$

$$P(G = 0, T = 1) = 0.9 \cdot 0.3 = 0.27,$$

$$P(G = 0, T = 0) = 0.9 \cdot 0.7 = 0.63.$$

Verificare:

$$0.08 + 0.02 + 0.27 + 0.63 = 1.$$

b) Rețeaua inversă $T \rightarrow G$

Calculăm marginalele lui T :

$$P(T = 0) = 0.02 + 0.63 = 0.65,$$

$$P(T = 1) = 0.08 + 0.27 = 0.35.$$

Apoi:

$$P(G = 0 \mid T = 0) = \frac{0.63}{0.65} = \frac{63}{65},$$

$$P(G = 1 \mid T = 0) = \frac{0.02}{0.65} = \frac{2}{65},$$

$$P(G = 0 \mid T = 1) = \frac{0.27}{0.35} = \frac{27}{35},$$

$$P(G = 1 \mid T = 1) = \frac{0.08}{0.35} = \frac{8}{35}.$$

4 Subiect 2 februarie 2024 - rezolvare dezvoltată

4.1 Problema 1 - Roboții Memo și Pic

Este aceeași familie de problema ca Melcul și Unicornul. Avem doi roboți într-un labirint 7×10 , cu obstacole, iar scopul este să ajungă pe aceeași celulă.

a) Reprezentare

$$s = (M_x, M_y, P_x, P_y).$$

Starea inițială, folosind coordonatele din rezolvarea de curs, este:

$$s_0 = (0, 0, 9, 6).$$

O stare finală este:

$$(a, b, a, b),$$

unde (a, b) este o poziție liberă.

Acțiunile sunt:

$$A = \{Nord, Sud, Est, Vest, stat\} \times \{Nord, Sud, Est, Vest, stat\}.$$

Exemplu:

$$Succ((M_x, M_y, P_x, P_y), (Nord, Sud)) = (M_x, M_y + 1, P_x, P_y - 1),$$

dacă ambele mișcări sunt valide.

b) Factor de ramificare

$$b_{max} = 5 \cdot 5 = 25.$$

c) Număr de stări

Labirintul are $7 \cdot 10 = 70$ celule și 20 obstacole. Deci fiecare robot poate ocupa una din:

$$70 - 20 = 50 \text{ poziții.}$$

Cum pot sta pe aceeași celulă:

$$|S| = 50 \cdot 50 = 2500.$$

d) Euristică admisibilă

$$h(s) = \left\lceil \frac{|M_x - P_x| + |M_y - P_y|}{2} \right\rceil.$$

Este admisibilă fiindcă ambii roboți se pot apropia simultan, iar ignorarea obstacolelor subestimează costul real.

4.2 Problema 2 - DFS, BFS, UCS, euristici

Din rezolvarea de curs, frontierele sunt:

a) DFS

$$\begin{aligned} &S \\ &S - A, S - G \\ &S - A - B, S - A - C, S - G \\ &S - A - B - D, S - A - C, S - G \\ &S - A - B - D - G, S - A - C, S - G. \end{aligned}$$

Soluție:

$$\boxed{S - A - B - D - G}.$$

b) BFS

$$S$$

$$S - A, S - G.$$

Soluție:

$$\boxed{S - G}.$$

c) UCSUCS alege drumul cu g minim:

$$S(0)$$

$$S - A(1), S - G(12)$$

$$S - A - B(4), S - A - C(2), S - G(12)$$

$$S - A - C - D(3), S - A - C - G(4), S - A - B(4), S - G(12)$$

$$S - A - C - D - G(6), S - A - C - G(4), S - A - B(4), S - G(12).$$

Primul scop scos cu cost minim este:

$$\boxed{S - A - C - G, \text{ cost} = 4}.$$

d) Admisibilitate și consistență pentru h_1, h_2

O euristica este admisibilă dacă:

$$h(n) \leq h^*(n).$$

Din rezolvare:

$$h^*(S) = 4, h^*(A) = 3, h^*(B) = 6, h^*(C) = 2, h^*(D) = 3, h^*(G) = 0.$$

 h_1 nu este admisibilă deoarece:

$$h_1(S) = 5 > h^*(S) = 4.$$

 h_2 este admisibilă.

Pentru consistență verificăm:

$$h(n) \leq c(n, n') + h(n').$$

 h_1 nu este consistentă deoarece:

$$h_1(S) = 5 \leq 1 + h_1(A) = 1 + 3 = 4$$

este fals. h_2 nu este consistentă deoarece:

$$h_2(S) = 4 \leq 1 + h_2(A) = 1 + 2 = 3$$

este fals.

4.3 Problema 3 - Minimax și Alpha-Beta**a) Valori Minimax**

Din rezolvare:

$$H = 3, \quad J = 9, \quad B = 4, \quad C = 3, \quad D = 3, \quad A = 4.$$

Interpretare: calculezi de la frunze spre rădăcina, alternând MAX și MIN.

b) Frunza minima si maxima

Frunza cu valoarea minima este:

$$\boxed{N = 2}.$$

Frunza cu valoarea maxima este:

$$\boxed{P = 9}.$$

c) Noduri retezate

Nodurile/frunzele retezate de Alpha-Beta sunt:

$$\boxed{J, O, P, L}.$$

d) Schimbarea ordinii de vizitare

- Schimba valoarea Minimax din radacina? **Nu.** Minimax este definit de arbore, nu de ordinea vizitarii.
- Schimba numarul de noduri retezate de Alpha-Beta? **Da.** O ordine mai buna produce limite α, β mai stranse mai devreme si poate taia mai mult.

4.4 Problema 4 - N-regine, forward checking si arc consistency

Avem $N = 5$ si reprezentarea:

$$(r_1, r_2, r_3, r_4, r_5),$$

unde r_i este linia reginei din coloana i .

Punem prima regina in coltul stanga-sus:

$$r_1 = 1.$$

a) Forward checking

Eliminam din domeniile celorlalte coloane linia 1 si diagonalele atacate de regina (1,1). Rezulta:

$$D_1 = \{1\},$$

$$D_2 = \{3, 4, 5\}, \quad D_3 = \{2, 4, 5\}, \quad D_4 = \{2, 3, 5\}, \quad D_5 = \{2, 3, 4\}.$$

b) Consistenta arcelor

Consistenta arcelor propaga mai departe constrangerile intre variabilele ramase. Dupa aplicarea algoritmului:

$$D_1 = \{1\}, \quad D_2 = \{3, 4, 5\}, \quad D_3 = \{2, 5\}, \quad D_4 = \{2, 5\}, \quad D_5 = \{3, 4, 5\}.$$

Diferenta fata de forward checking este ca AC-3 poate elimina valori care nu sunt direct atacate de prima regina, dar care nu mai au suport in domeniile altor variabile.

5 Subiect 4 februarie 2025 - rezolvare dezvoltată

5.1 Problema 1 - Robot care trebuie să treacă prin G_1 și G_2

Robotul se află într-un grid 3×4 cu o celulă obstacol. Sunt 11 poziții libere numerotate. Startul este poziția 3, iar scopurile intermediare sunt G_1 în poziția 2 și G_2 în poziția 6.

a) Reprezentare

Pentru că trebuie să știm dacă robotul a trecut prin G_1 și G_2 , starea nu poate fi doar poziția. Folosim:

$$s = (p, v_1, v_2),$$

unde:

- p este poziția curentă a robotului;
- $v_1 = 1$ dacă G_1 a fost vizitat, altfel 0;
- $v_2 = 1$ dacă G_2 a fost vizitat, altfel 0.

Starea inițială:

$$s_0 = (3, 0, 0).$$

O stare scop este orice stare:

$$(p, 1, 1),$$

unde p este orice poziție liberă. Nu contează unde se oprește robotul după ce a trecut prin ambele scopuri.

b) Câte stări neterminale există?

Avem 11 poziții și 4 combinații posibile pentru (v_1, v_2) :

$$(0, 0), (1, 0), (0, 1), (1, 1).$$

Total brut:

$$11 \cdot 4 = 44.$$

Stările terminale sunt cele cu $(v_1, v_2) = (1, 1)$, adică 11 stări. Deci stările neterminale sunt:

$$\boxed{44 - 11 = 33}.$$

c) Arbore de căutare de adâncime 2

Pornim din:

$$(3, 0, 0).$$

Din poziția 3 putem merge la 2, 4, 6:

$$(2, 1, 0), \quad (4, 0, 0), \quad (6, 0, 1).$$

La nivelul 2:

- din $(2, 1, 0)$ putem ajunge la $(1, 1, 0)$ și $(3, 1, 0)$;
- din $(4, 0, 0)$ putem ajunge la $(3, 0, 0)$ și $(7, 0, 0)$;
- din $(6, 0, 1)$ putem ajunge la $(3, 0, 1)$, $(7, 0, 1)$ și $(10, 0, 1)$.

Starea $(3, 0, 0)$ re apare în arbore, deci este o stare repetată și trebuie încercuită.

5.2 Problema 2 - DFS, BFS, Greedy, A* și consistentă

Graful are muchii:

$$\begin{aligned} S &\rightarrow A(3), \quad S \rightarrow D(2), \quad A \rightarrow B(5), \quad A \rightarrow G(10), \\ D &\rightarrow B(1), \quad D \rightarrow E(4), \quad B \rightarrow C(2), \quad B \rightarrow E(1), \quad C \rightarrow G(4), \quad E \rightarrow G(3). \end{aligned}$$

Euristici:

$$h(S) = 7, \quad h(A) = 9, \quad h(B) = 4, \quad h(C) = 2, \quad h(D) = 5, \quad h(E) = 3, \quad h(G) = 0.$$

a) DFS

Ordine lexicografica: din S alegem A inainte de D .

$$\begin{aligned}
 &S \\
 &S - A, S - D \\
 &S - A - B, S - A - G, S - D \\
 &S - A - B - C, S - A - B - E, S - A - G, S - D \\
 &S - A - B - C - G, S - A - B - E, S - A - G, S - D.
 \end{aligned}$$

Solutie DFS:

$$\boxed{S - A - B - C - G}.$$

b) BFS eficient

$$\begin{aligned}
 &S \\
 &S - A, S - D.
 \end{aligned}$$

Expandand $S - A$, se genereaza $S - A - G$, scop la adancime 2. Solutie:

$$\boxed{S - A - G}.$$

c) Greedy

Greedy alege dupa h minim.

$$S(7) \Rightarrow A(9), D(5) \Rightarrow D.$$

Din D :

$$B(4), E(3) \Rightarrow E.$$

Din E :

$$G(0).$$

Solutie:

$$\boxed{S - D - E - G}.$$

d) A*

Calculam $f = g + h$.

$$S : 0 + 7 = 7.$$

Dupa expandarea lui S :

$$S - A : 3 + 9 = 12, \quad S - D : 2 + 5 = 7.$$

Alegem D .

$$S - D - B : 3 + 4 = 7, \quad S - D - E : 6 + 3 = 9, \quad S - A : 12.$$

Alegem B .

$$S - D - B - C : 5 + 2 = 7, \quad S - D - B - E : 4 + 3 = 7, \quad S - D - E : 9, \quad S - A : 12.$$

Alegem C lexicografic.

$$S - D - B - C - G : 9 + 0 = 9, \quad S - D - B - E : 7, \quad S - D - E : 9, \quad S - A : 12.$$

Alegem E de pe drumul $S - D - B - E$.

$$S - D - B - E - G : 7 + 0 = 7.$$

Primul scop scos este:

$$\boxed{S - D - B - E - G, \text{ cost} = 7}.$$

e) Consistența euristicii

Verificăm $h(n) \leq c(n, n') + h(n')$ pe fiecare arc:

- $S \rightarrow A: 7 \leq 3 + 9 = 12;$
- $S \rightarrow D: 7 \leq 2 + 5 = 7;$
- $A \rightarrow B: 9 \leq 5 + 4 = 9;$
- $A \rightarrow G: 9 \leq 10 + 0 = 10;$
- $D \rightarrow B: 5 \leq 1 + 4 = 5;$
- $D \rightarrow E: 5 \leq 4 + 3 = 7;$
- $B \rightarrow C: 4 \leq 2 + 2 = 4;$
- $B \rightarrow E: 4 \leq 1 + 3 = 4;$
- $C \rightarrow G: 2 \leq 4 + 0 = 4;$
- $E \rightarrow G: 3 \leq 3 + 0 = 3.$

Toate sunt adevărate, deci:

$$\boxed{h \text{ este consistentă.}}$$

5.3 Problema 3 - Minimax cu valori lipsa E și K

Arborele are rădăcina A de tip MAX. Copiii B, C, D sunt de tip MIN. Nodul H este de tip MAX.

a) Valorile H și C

$$H = \max(N = 3, O = 5) = 5.$$

$$C = \min(H = 5, I = 7, J = 4) = 4.$$

Deci:

$$\boxed{H = 5, \quad C = 4.}$$

b) Alegem valoarea lui E astfel încât MAX să aleagă B

$$B = \min(E, 6, 5) = \min(E, 5).$$

Știm că $C = 4$ și $D \leq 1$ deoarece $D = \min(K, 1, 2)$. Ca MAX să aleagă B , vrem $B > 4$. Este suficient să alegem:

$$E \geq 5.$$

De exemplu:

$$\boxed{E = 5}.$$

Atunci:

$$B = \min(5, 6, 5) = 5,$$

și MAX alege B , deoarece $B = 5 > C = 4$.

c) Alegem K ca să retezăm L și M când $E = 1$

Dacă $E = 1$, atunci:

$$B = \min(1, 6, 5) = 1.$$

După evaluarea lui C , rădăcina MAX va avea:

$$\alpha = \max(1, 4) = 4.$$

La nodul D de tip MIN, primul copil este K . Pentru ca după evaluarea lui K să se taie L și M , avem nevoie de:

$$K \leq \alpha = 4.$$

Alegem, de exemplu:

$$\boxed{K = 3}.$$

Atunci la D obținem $\beta = 3 \leq \alpha = 4$, deci restul copiilor L, M se retează.

5.4 Problema 4 - Sudoku ca PSC

a) De ce alegem variabila cea mai constransă?

Euristica MRV - Minimum Remaining Values - alege variabila cu cele mai puține valori posibile rămase. Este bună deoarece găsește mai devreme blocajele. Dacă o variabilă are un singur candidat, trebuie completată imediat; dacă acel candidat duce la conflict, backtracking-ul revine devreme, fără să piardă timp pe ramuri inutile.

b) Aplicare pe Sudoku

Din grila dată, unele celule au un singur candidat. De exemplu celula $E6$ are candidatul unic:

$$E6 = 4.$$

Justificare: pe linia E , coloana 6 și blocul central lipsesc mai multe valori, dar intersecția valorilor permise lasă doar 4. Deci o alegere corectă după MRV este:

$$\boxed{E6 = 4}.$$

Și $E7 = 1$, $I4 = 4$ sunt tot celule cu un singur candidat; oricare poate fi aleasă dacă regula de departajare nu este specificată.

5.5 Problema 5 - Reteaua Bayesiană Wet Grass

Graful este:

$$Cloud \rightarrow Rain, \quad Cloud \rightarrow Sprinkle, \quad Rain \rightarrow WetGrass, \quad Sprinkle \rightarrow WetGrass.$$

a) Formula distribuției comune

$$P(C, R, S, W) = P(C)P(R | C)P(S | C)P(W | S, R).$$

b) Calcul $P(C = 1, R = 1, S = 0, W = 1)$

Din tabele:

$$P(C = 1) = 0.5,$$

$$P(R = 1 | C = 1) = 0.8,$$

$$P(S = 0 | C = 1) = 0.9,$$

$$P(W = 1 | S = 0, R = 1) = 0.9.$$

Deci:

$$P(C = 1, R = 1, S = 0, W = 1) = 0.5 \cdot 0.8 \cdot 0.9 \cdot 0.9 = 0.324.$$

c) Este Rain independent de Sprinkle dat Cloud?

Da. În graf avem structura:

$$Rain \leftarrow Cloud \rightarrow Sprinkle.$$

Aceasta este o cauză comună. Observarea cauzei comune $Cloud$ blochează dependența dintre efecte. Deci:

$$\boxed{Rain \perp Sprinkle | Cloud}.$$

Altfel spus:

$$P(R, S | C) = P(R | C)P(S | C).$$

6 Model subiect decembrie 2023 - rezolvare dezvoltată

6.1 Problema 1 - Robot pe drum alb/negru

Avem un drum de N patrate cunoscute. Robotul porneste din cel mai din stanga și trebuie să ajungă după ultimul patrat. Dacă este pe alb, poate sări 1 sau 2 patrate. Dacă este pe negru, poate sări 1 sau 4 patrate.

a) Stare, start, scop

O stare este poziția robotului:

$$s = i,$$

unde $i \in \{1, \dots, N+1\}$. Poziția $N+1$ înseamnă că robotul a ieșit după ultimul patrat.

Starea inițială:

$$s_0 = 1.$$

Starea scop:

$$s = N+1 \quad \text{sau} \quad s > N.$$

b) Acțiuni

Dacă patratul i este alb:

$$A(i) = \{+1, +2\}.$$

Dacă patratul i este negru:

$$A(i) = \{+1, +4\}.$$

c) Succesor

$$\text{Succ}(i, +k) = i + k.$$

Dacă $i + k > N$, succesorul este starea scop $N+1$. Costul fiecărei acțiuni este 1, deoarece vrem număr minim de mutări.

6.2 Problema 2 - Graf cu DFS, Greedy și A*

Muchiile relevante:

$$\begin{aligned} S &\rightarrow A(6), S \rightarrow B(5), S \rightarrow C(10), A \rightarrow E(4), \\ B &\rightarrow E(6), B \rightarrow D(7), C \rightarrow D(6), D \rightarrow F(6), E \rightarrow F(4), F \rightarrow G(3). \end{aligned}$$

Euristici:

$$h(S) = 17, h(A) = 10, h(B) = 13, h(C) = 4, h(D) = 2, h(E) = 4, h(F) = 1, h(G) = 0.$$

a) DFS

Ordine lexicografică:

$$S \rightarrow A \rightarrow E \rightarrow F \rightarrow G.$$

Soluție:

$$\boxed{S - A - E - F - G}.$$

b) Greedy

Greedy alege după h minim:

$$\begin{aligned} S &: A(10), B(13), C(4) \Rightarrow C. \\ C &\Rightarrow D(2), \quad D \Rightarrow F(1), \quad F \Rightarrow G(0). \end{aligned}$$

Soluție:

$$\boxed{S - C - D - F - G}.$$

c) A*

Calculăm $f = g + h$:

$$S : 0 + 17 = 17.$$

Dupa expandarea lui S :

$$S - A : 6 + 10 = 16, \quad S - B : 5 + 13 = 18, \quad S - C : 10 + 4 = 14.$$

Alegem $S - C$ și generăm:

$$S - C - D : 16 + 2 = 18.$$

Frontiera:

$$S - A(16), \quad S - B(18), \quad S - C - D(18).$$

Alegem $S - A$:

$$S - A - E : 10 + 4 = 14.$$

Alegem $S - A - E$:

$$S - A - E - F : 14 + 1 = 15.$$

Alegem $S - A - E - F$:

$$S - A - E - F - G : 17 + 0 = 17.$$

Primul scop scos este:

$$\boxed{S - A - E - F - G, \text{ cost} = 17}.$$

6.3 Problema 3 - Conecteaza-3**a) Factor maxim de ramificare**

Sunt 4 coloane. Într-un moment în care toate coloanele au loc disponibil, jucătorul poate alege oricare coloana. Deci:

$$\boxed{b_{max} = 4}.$$

Dacă o coloană este plină, nu mai este succesor valid.

b) Arborele din starea dată

Din figura 3b, coloanele 1 și 2 sunt pline, iar jucătorul 0 este la mutare. Coloanele valide sunt 3 și 4. Radacina are doi succesori:

$$0 \text{ joacă în col. 3} \quad \text{sau} \quad 0 \text{ joacă în col. 4.}$$

Dupa aceea joacă X , apoi eventual 0 pentru completarea tablei.

c) Valori Minimax

Considerăm $X = MAX$, $0 = MIN$. Din analiza ramurilor:

- dacă 0 joacă în coloana 3, X poate evita infrangerea și se ajunge la remiza, valoare 0;
- dacă 0 joacă în coloana 4, jocul ajunge de asemenea la remiza cu joc optim, valoare 0.

Deci radacina are valoarea:

$$\boxed{0}.$$

d) Alpha-Beta și reordonare

Alpha-Beta poate rețeza mai mult dacă la nodurile MAX evaluăm întâi mutările cu valori mari, iar la nodurile MIN evaluăm întâi mutările cu valori mici. În acest arbore mic, rețezările depind strict de ordinea în care sunt puse cele două coloane posibile. Reordonarea copiilor nu schimbă valoarea Minimax, doar numărul de noduri vizitate.

6.4 Problema 4 - Euristica PSC în Sudoku

a) De ce alegem variabila cea mai constransă și valoarea cea mai puțin constrângătoare?

Variabila cea mai constransă are cele mai puține valori posibile. Alegând-o devreme, descoperim rapid dacă ramura curentă este imposibilă. Valoarea cea mai puțin constrângătoare este bună deoarece lasă cât mai multe opțiuni pentru restul variabilelor, reducând riscul de blocaj ulterior.

b) Aplicare pe Sudoku

Din grila, celulele cu un singur candidat includ $E6 = 4$, $E7 = 1$, $I4 = 4$. Alegem, de exemplu:

$$\boxed{E6 = 4}.$$

Justificare: candidatul se obține prin intersecția valorilor permise de linia E , coloana 6 și blocul central.

7 Curs 14 - D-separare și rețele Bayesiene

7.1 Cele trei triplete fundamentale

7.1.1 1. Lant cauzal

Forma:

$$X \rightarrow Y \rightarrow Z.$$

Dacă Y nu este observat, drumul este activ și X poate influența Z . Dacă Y este observat, drumul este blocat:

$$X \perp Z \mid Y.$$

Intuitiv: dacă știm deja cauza intermediară, informația despre X nu mai adaugă nimic despre Z pe acel drum.

7.1.2 2. Cauza comună

Forma:

$$X \leftarrow Y \rightarrow Z.$$

Dacă Y nu este observat, drumul este activ. Dacă Y este observat, drumul este blocat:

$$X \perp Z \mid Y.$$

Exemplu: ploaia poate explica atât umbrela, cât și traficul. Dacă știm că plouă, umbrela și traficul devin independente pe acea cale.

7.1.3 3. Efect comun / collider

Forma:

$$X \rightarrow Y \leftarrow Z.$$

Aici regula este inversă:

- dacă Y nu este observat și niciun descendent al lui Y nu este observat, drumul este blocat;
- dacă Y este observat sau un descendent al lui este observat, drumul devine activ.

Exemplu: ploaia și concertul pot cauza trafic. Dacă observăm traficul, apar dependențe între posibilele cauze.

7.2 Algoritm de D-separare

Vrem să verificăm:

$$X_i \perp X_j \mid E,$$

unde E este mulțimea de variabile observate.

1. Considerăm toate drumurile neorientate dintre X_i și X_j .
2. Pentru fiecare drum, îl împărțim în triplete consecutive.
3. Un drum este activ dacă toate tripletele lui sunt active.
4. Dacă există cel puțin un drum activ, independența nu este garantată.
5. Dacă toate drumurile sunt inactive, variabilele sunt d-separate, deci independența condiționată este garantată.

7.3 Exemplu: $P \rightarrow T \leftarrow C, T \rightarrow A$

Variabile:

- P = ploaie;
- C = concert;
- T = trafic;
- A = accident.

1. Este $P \perp C$?

Drumul este:

$$P \rightarrow T \leftarrow C.$$

T este collider și nu este observat. Drumul este blocat. Deci:

$$\boxed{P \perp C}.$$

2. Este $P \perp C \mid T$?

Acum observăm colliderul T . Colliderul se activează. Drumul devine activ, deci:

$$\boxed{P \not\perp C \mid T}.$$

3. Este $P \perp C \mid A$?

A este descendent al colliderului T . Observarea unui descendent al colliderului activează drumul. Deci:

$$\boxed{P \not\perp C \mid A}.$$

7.4 Exemplu extins: $P_s \rightarrow P, P \rightarrow U, P \rightarrow T, C \rightarrow T, T \rightarrow A$

Variabile:

- P_s = presiune scăzută;
- P = ploaie;
- U = umbrelă;
- C = concert;
- T = trafic;
- A = accident.

1. $P_s \perp A \mid T$

Drumul relevant:

$$P_s \rightarrow P \rightarrow T \rightarrow A.$$

Observăm T , care este nod de tip lant pe drum. Lantul este blocat, deci:

$$\boxed{P_s \perp A \mid T}.$$

2. $P_s \perp C$

Drumul:

$$P_s \rightarrow P \rightarrow T \leftarrow C.$$

T este collider neobservat, deci blochează drumul:

$$\boxed{P_s \perp C}.$$

3. $P_s \perp C \mid T$

Observăm colliderul T , deci drumul se activează:

$$\boxed{P_s \not\perp C \mid T}.$$

4. $P_s \perp C \mid A$

A este descendent al colliderului T . Observarea lui A activează colliderul:

$$\boxed{P_s \not\perp C \mid A}.$$

Important: La d-separare, cel mai frecvent greșit este colliderul. Pentru lanț și cauză comună, observarea nodului din mijloc blochează. Pentru collider, observarea nodului din mijloc sau a unui descendent activează.

8 Fisa finala pentru examen

Formulare problema

- Stare = informatia necesara pentru a continua problema.
- Doi agenti = stare cu pozitiile ambilor agenti.
- Obiecte/scopuri intermediare = adaugi in stare ce a fost colectat/vizitat.
- b_{max} = produsul actiunilor independente ale agentilor.
- Numar de stari = produsul posibilitatilor pentru componentele starii.

Cautare

- DFS = adancime, stiva, nu garanteaza optim.
- BFS = niveluri, coada, optim pentru costuri egale.
- UCS = minim $g(n)$, optim pentru costuri pozitive.
- Greedy = minim $h(n)$, nu garanteaza optim.
- A^* = minim $f(n) = g(n) + h(n)$.

Euristici

- Admisibila: $h(n) \leq h^*(n)$.
- Consistenta: $h(n) \leq c(n, n') + h(n')$ pentru orice arc.
- Consistenta implica admisibilitate.
- Pentru a arata ca nu e admisibila, gasesti un singur nod cu $h(n) > h^*(n)$.
- Pentru a arata ca nu e consistenta, gasesti un singur arc unde inegalitatea pica.

Minimax si Alpha-Beta

- MAX ia maximul copiilor.
- MIN ia minimul copiilor.
- Alpha-Beta nu modifica valoarea Minimax.
- Tai cand $\alpha \geq \beta$.
- Ordonare buna: valori mari prima data la MAX, valori mici prima data la MIN.

PSC/CSP

- $PSC = (X, D, C)$.
- Stare = asignare partiala.
- Scop = asignare completa si consistenta.
- Forward checking = dupa asignare elimini valori din domeniile vecinilor.
- Arc consistency = fiecare valoare dintr-un domeniu trebuie sa aiba suport in domeniul vecinului.
- MRV = aleg variabila cu cele mai putine valori posibile.

Rețele Bayesiene

- Factoring: $P(X_1, \dots, X_n) = \prod_i P(X_i \mid \text{Parinti}(X_i))$.
- Lant/cauza comuna: observarea nodului din mijloc blocheaza.
- Collider: neobservat blocheaza; observat sau descendent observat activeaza.

Surse folosite

Material sintetizat din cursurile si subiectele incarcate: Curs14, modelul 2023, subiectele 2024/2025/2026, rezolvarea oficiala si rezolvarile de seminar KR.